

Documentación Técnica Completa: Sistema de Gestión de Estacionamientos (Python + Flet + SQLite)

1. Descripción General del Proyecto

El **Sistema de Gestión de Estacionamientos** es una aplicación de escritorio multiplataforma estructurada como una **Single Page Application (SPA)**. Permite la administración integral de predios de estacionamiento, la gestión de plazas/lugares según el tipo de vehículo, la reserva en tiempo real por parte de conductores, y el control financiero y de recaudación por parte de los administradores.

La aplicación resuelve de forma automatizada:

- **Autenticación y Autorización:** Registro diferenciado de usuarios por rol (**admin** y **conductor**).
 - **Multipredio/Multi-estacionamiento:** Selección de distintos predios o sucursales de estacionamiento.
 - **Control de Plazas/Lugares:** ABM (Alta, Baja y Modificación) de lugares, indicando tipo de vehículo (**auto**, **moto**, **camioneta**), número y precio por hora.
 - **Flujo de Reservas y Ocupación:** Reserva inmediata de lugares disponibles, cambio automático de estado a ocupado y cancelación de reservas.
 - **Facturación y Cobro Automatizado:** Cálculo exacto del tiempo de estadía en horas (redondeando hacia arriba la fracción de hora transcurrida) y cobro total al finalizar la reserva.
 - **Métricas Financieras:** Panel para el administrador con reporte de ingresos acumulados (Hoy, Este Mes y Total Histórico) e historial detallado de cobros y salidas.
-

2. Tecnologías y Librerías Utilizadas

Tecnología / Librería	Tipo	Propósito y Forma de Uso
Python 3.x	Lenguaje Base	Lenguaje principal de programación para toda la lógica de backend y frontend.

Tecnología / Librería	Tipo	Propósito y Forma de Uso
Flet (<code>import flet as ft</code>)	Framework UI	Framework basado en Flutter que permite construir la interfaz gráfica (UI) reactiva, gestionar eventos, ruteo SPA, estados y controles visuales.
SQLite3 (<code>import sqlite3</code>)	Base de Datos	Motor de base de datos relacional embebido. Administra la persistencia de datos local sin requerir un servidor externo.
hashlib	Seguridad	Generación de hashes unidireccionales (SHA-256) para encriptar contraseñas y DNI de los usuarios antes de almacenarlos.
datetime	Módulo Estándar	Captura de marcas de tiempo (<code>timestamp</code>) para fechas de reserva, inicio y fin de estadía.
math	Módulo Estándar	Utilizado específicamente para <code>math.ceil()</code> , permitiendo redondear la fracción de tiempo consumido a la hora entera superior cobrable.

3. Arquitectura del Sistema

El proyecto sigue una arquitectura modular desacoplada en dos componentes fundamentales:

Aplicación de Estacionamiento/

|

|— database.py --> CAPA DE DATOS Y LÓGICA DE NEGOCIO (DAL / DAO)

| Conexión a SQLite, DDL (tablas), DML (queries SQL),
| encriptación SHA-256, reglas de cobro y recaudación.
|
└─ main.py --> CAPA DE PRESENTACIÓN E INTERFAZ DE USUARIO (UI / Flet)

Ruteo SPA, gestión de sesión en memoria, formularios,

eventos de interfaz, alertas (SnackBar) y vistas.

4. Estructura y Esquema de la Base de Datos (**database.py**)

La base de datos relacional se compone de 4 tablas principales vinculadas mediante restricciones de clave foránea (**FOREIGN KEY**):

4.1. Diagrama de Tablas

1. **usuarios**:

- **id** (INTEGER PRIMARY KEY AUTOINCREMENT)
- **username** (TEXT UNIQUE NOT NULL)
- **password** (TEXT NOT NULL - Hash SHA-256)
- **nombre** (TEXT NOT NULL)
- **rol** (TEXT CHECK **rol** IN ('admin', 'conductor'))
- **email** (TEXT)
- **dni** (TEXT - Hash SHA-256)

2. **estacionamientos**:

- **id** (INTEGER PRIMARY KEY AUTOINCREMENT)
- **nombre** (TEXT NOT NULL)
- **direccion** (TEXT)

3. **lugares**:

- **id** (INTEGER PRIMARY KEY AUTOINCREMENT)
- **numero** (INTEGER NOT NULL)

- `tipo_vehiculo` (TEXT CHECK `tipo_vehiculo` IN ('auto', 'moto', 'camioneta'))
- `precio_hora` (REAL NOT NULL)
- `disponible` (INTEGER DEFAULT 1)
- `estacionamiento_id` (INTEGER NOT NULL, FK -> `estacionamientos(id)` ON DELETE CASCADE)
- `UNIQUE(numero, estacionamiento_id)`

4. **reservas:**

- `id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `lugar_id` (INTEGER NOT NULL, FK -> `lugares(id)` ON DELETE CASCADE)
- `usuario_id` (INTEGER NOT NULL, FK -> `usuarios(id)` ON DELETE CASCADE)
- `fecha_reserva` (TEXT NOT NULL)
- `fecha_fin` (TEXT)
- `monto_total` (REAL DEFAULT 0.0)
- `estado` (TEXT DEFAULT 'activa' CHECK `estado` IN ('activa', 'finalizada', 'cancelada'))

5. Explicación Detallada de Funciones

5.1. Funciones en `database.py`

- **`get_connection()`**: Abre la conexión con `estacionamiento.db`, activa explícitamente `PRAGMA foreign_keys = ON` para garantizar integridad referencial y asigna `conn.row_factory = sqlite3.Row` para acceder a las columnas por nombre.
- **`init_db()`**: Inicializa las tablas si no existen e inserta un usuario administrador por defecto (`admin / admin123`) y estacionamientos iniciales de ejemplo si la tabla está vacía.
- **`preparar_tabla_usuarios()` y `preparar_tabla_facturacion()`**: Funciones de migración defensiva que ejecutan `ALTER TABLE` para agregar campos como `email`, `dni`, `fecha_fin` y `monto_total` sin romper datos existentes.
- **`encriptar_dato(texto)`**: Recibe una cadena de texto y devuelve su representación hash hexadecimal SHA-256 de 64 caracteres.
- **`registrar_usuario(...)`**: Aplica validaciones defensivas (DNI numérico de 8 dígitos, contraseña $\geq$ 8 caracteres, verificación de usuario/correo duplicado) e inserta el usuario guardando las claves encriptadas.

- **validar_login(username, password):** Compara el hash de la contraseña ingresada con la base de datos y retorna el registro del usuario si las credenciales coinciden.
 - **finalizar_reserva_y_cobrar(reserva_id):**
 1. Obtiene la fecha de inicio de la reserva y el precio por hora del lugar.
 2. Calcula los segundos transcurridos hasta el momento actual (`datetime.now()`).
 3. Convierte el tiempo a horas mediante `math.ceil(segundos / 3600.0)`, asegurando un cobro mínimo de 1 hora.
 4. Calcula `monto_total = horas_cobradas * precio_hora`.
 5. Actualiza el estado de la reserva a 'finalizada', guarda el monto y libera el lugar marcándolo como `disponible = 1`.
 - **obtener_resumen_recaudacion(estacionamiento_id):** Ejecuta sumatorias (`SUM(monto_total)`) filtrando las reservas finalizadas del día actual (Hoy), del mes actual (Este Mes) y de todo el registro histórico (Total).
-

5.2. Arquitectura y Vistas en `main.py`

- **Gestión de Sesión:** Se mantiene un diccionario en memoria `sesion = {"usuario": None, "estacionamiento_id": None}` accesible globalmente en las vistas. Al iniciar sesión, se remueve el hash de la contraseña por seguridad (`usuario_seguro.pop("password", None)`).
- **Mecanismo de Ruteo (SPA):** La función `route_change(e)` escucha cambios de ruta en la aplicación (`page.route`), limpia las vistas activas (`page.views.clear()`) y renderiza únicamente la vista correspondiente según el estado de la sesión y el rol del usuario:
 - `/login`: Pantalla de inicio de sesión.
 - `/registro`: Formulario de creación de cuenta para conductores.
 - `/estacionamientos`: Selector de predios de estacionamiento.
 - `/admin`: Panel de control restringido para administradores.
 - `/conductor`: Panel principal para conductores.

Detalle de Vistas en `main.py`:

2. **Vista Login (`vista_login`):** Captura usuario y contraseña, efectúa la validación con la base de datos y redirige a `/estacionamientos`.
3. **Vista Registro (`vista_registro`):** Formulario para registrar conductores recopilando Nombre, Usuario, Email, DNI y Contraseña.
4. **Vista Lista de Estacionamientos (`vista_estacionamientos`):** Renderiza tarjetas dinámicas para cada predio mostrando su nombre, dirección y la cantidad de lugares disponibles versus totales.
5. **Vista Panel Administrador (`vista_admin`):**

- Muestra las tarjetas con las métricas de recaudación (Hoy, Mes, Total).
 - Formulario interactivo para agregar y editar lugares (Número, Tipo de Vehículo, Precio por Hora).
 - Lista de lugares cargados con acciones para Editar y Eliminar.
 - Tabla con el historial completo de cobros y salidas.
6. **Vista Panel Conductor (`vista_conductor`):**
- Filtro desplegable por tipo de vehículo (`todos`, `auto`, `moto`, `camioneta`).
 - Grilla de lugares disponibles con botón directo para "Reservar".
 - Sección "Mis reservas" donde el conductor puede ver el estado de sus reservas, cancelar una reserva activa o presionar "**Finalizar y Pagar**" para cerrar la estadía y ver el total abonado.
-

6. Patrones de Seguridad e Integridad Aplicados

- **Prevención de Inyección SQL:** Todas las consultas en `database.py` emplean consultas preparadas y parametrizadas utilizando el marcador `?`.
- **Hash de Datos Sensibles:** Uso del algoritmo SHA-256 para almacenamiento seguro de contraseñas y números de DNI.
- **Limpieza de Datos de Sesión:** Eliminación del campo `password` del diccionario de sesión en memoria inmediatamente después del inicio de sesión.
- **Defensive Database Pattern:** Integridad de claves foráneas con `PRAGMA foreign_keys = ON` y manejo de borrados en cascada (`ON DELETE CASCADE`).
- **Sanitización de Entradas:** Uso sistemático de `.strip()` en todas las cadenas ingresadas por el usuario para eliminar espacios no deseados.